Université Abdelmalek Essaâdi
L'Ecole Nationale des Sciences Appliquées d'Al-Hoceima
Filière :Transformation Digitale et Intelligence Artificielle

Compte Rendu de TP5:
Gestion des Produits avec Hibernate

Encadré par : Pr. M. CHERRADI
Réalisé par : Nada EL-KHAMLICHI
Module : Applications Web Avancées avec Java et Spring

Année universitaire : 2025 / 2026

1. Introduction :

Dans le cadre du module Applications Web Avancées avec Java et Spring, ce travail pratique a pour objectif la conception et la réalisation d'une application web de gestion des produits, intégrant un système d'authentification et de contrôle d'accès basé sur les rôles.

Ce projet s'inscrit dans une démarche d'apprentissage des architectures logicielles modernes, en particulier le modèle **MVC2**, ainsi que l'utilisation du framework **Hibernate** pour la gestion de la persistance des données. Dans cette première version, Hibernate est exploité à travers une configuration basée sur des fichiers XML, permettant une séparation explicite entre la logique métier et les mécanismes de mapping objet-relationnel.

L'application développée vise ainsi à simuler un système d'information simplifié, dans lequel différents types d'utilisateurs interagissent avec une base de données relationnelle via une interface web dynamique.

2. Objectifs du travail :

L'objectif principal de ce TP est de mettre en œuvre, dans un contexte pratique, les concepts fondamentaux du développement d'applications web d'entreprise.

Plus précisément, ce travail vise à :

- maîtriser l'architecture MVC2 en séparant clairement les responsabilités entre les différentes couches de l'application ;
- implémenter un mécanisme de persistance des données à l'aide de Hibernate en configuration XML ;
- concevoir et manipuler une base de données relationnelle sous MySQL ;
- développer un système d'authentification sécurisé basé sur des sessions HTTP ;
- gérer les droits d'accès en fonction des rôles des utilisateurs (administrateur ou utilisateur standard).

Ce TP constitue ainsi une étape essentielle dans la compréhension des systèmes web distribués et de leur organisation interne.

3. Architecture de l'application :

L'application a été conçue selon le modèle architectural **MVC2 (Model–View–Controller)**, largement utilisé dans le développement des applications web Java. Cette architecture permet de structurer le projet de manière modulaire, facilitant ainsi sa maintenance et son évolution.

3.1 Couche Modèle (Model) :

La couche modèle regroupe l'ensemble des entités métier manipulées par l'application. Dans ce projet, trois entités principales ont été définies :

- l'entité `User`, représentant les utilisateurs du système ;
- l'entité `Role`, définissant les différents niveaux d'accès ;
- l'entité `Produit`, représentant les éléments gérés par l'application.

Dans cette version, la correspondance entre ces classes Java et les tables de la base de données est assurée à l'aide de fichiers de mapping XML (`.hbm.xml`). Cette approche permet de configurer explicitement les relations entre objets et tables, tout en conservant une séparation nette entre le code Java et la configuration de persistance.

3.2 Couche d'accès aux données (DAO) :

La couche DAO constitue l'interface entre l'application et la base de données. Elle encapsule les opérations de persistance en utilisant Hibernate, permettant ainsi d'abstraire les détails liés aux requêtes SQL.

Chaque entité possède une classe DAO dédiée (par exemple `UserDaoImpl`, `ProduitDaoImpl`, etc.), dans laquelle sont implémentées les opérations fondamentales telles que :

- l'insertion de nouvelles données ;
- la récupération d'informations ;
- la mise à jour ;
- la suppression.

Cette couche joue un rôle essentiel dans la structuration du projet, en isolant la logique d'accès aux données du reste de l'application.

3.3 Couche Service :

La couche service assure l'implémentation de la logique métier. Elle se situe entre le contrôleur et la couche DAO, et permet de centraliser les traitements nécessaires au bon fonctionnement de l'application.

Elle est notamment responsable de :

- la validation des données saisies par l'utilisateur ;
- la vérification des règles métier (unicité des identifiants, cohérence des opérations, etc.) ;
- la coordination des différentes opérations DAO.

Cette séparation permet de rendre le code plus lisible, plus maintenable et plus évolutif.

3.4 Couche Contrôleur (Controller) :

Le contrôleur est implémenté à travers une servlet principale nommée `ActionServlet`. Cette dernière constitue le point d'entrée unique de l'application.

Elle reçoit les requêtes HTTP émises par les utilisateurs, identifie l'action demandée (connexion, ajout de produit, suppression, etc.) et délègue le traitement à la couche

service. Une fois le traitement effectué, le contrôleur redirige l'utilisateur vers la vue appropriée.

Ce mécanisme centralisé permet de simplifier la gestion des flux de navigation au sein de l'application.

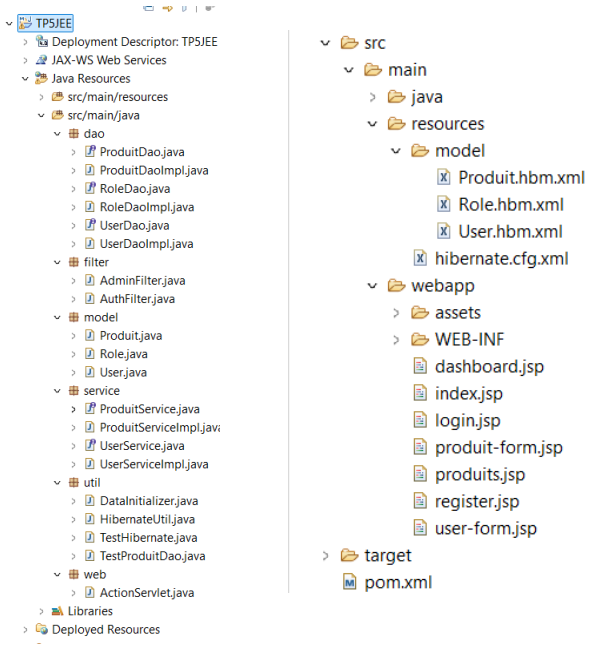
3.5 Couche Vue (View) :

La couche vue est réalisée à l'aide de pages JSP, permettant de générer dynamiquement le contenu HTML affiché à l'utilisateur.

Parmi les principales pages développées, on peut citer :

- la page de connexion (`login.jsp`) ;
- la page d'inscription (`register.jsp`) ;
- la page d'affichage des produits (`produits.jsp`) ;
- le formulaire d'ajout et de modification (`produit-form.jsp`).

Ces vues interagissent avec le contrôleur via des formulaires HTTP et affichent les données transmises sous forme d'attributs de requête.



4. Configuration de Hibernate (version XML) :

Dans la première version, Hibernate est configuré à travers des fichiers XML, offrant un contrôle explicite sur le mapping objet-relationnel.

Le fichier principal `hibernate.cfg.xml` contient les paramètres de connexion à la base de données ainsi que les références vers les fichiers de mapping.

Chaque entité est associée à un fichier `.hbm.xml`, dans lequel sont définis :

- le nom de la table correspondante ;
- les colonnes associées aux attributs de la classe ;
- les clés primaires ;
- les relations entre entités.

Bien que cette approche soit plus verbeuse que les annotations, elle permet une meilleure visibilité sur la configuration et facilite la compréhension des mécanismes internes de Hibernate.

5. Configuration de Hibernate (version Annotations) :

Dans une seconde étape de ce travail, une version alternative de l'application a été développée en utilisant la configuration de Hibernate basée sur les **annotations Java**.

Contrairement à l'approche précédente, qui repose sur des fichiers XML externes, cette méthode consiste à intégrer directement les informations de mapping objet-relationnel au sein des classes du modèle. Ainsi, chaque entité est enrichie par un ensemble d'annotations permettant de définir sa correspondance avec la base de données.

Les principales annotations utilisées dans ce projet sont :

- `@Entity`, permettant de déclarer une classe comme entité persistante ;
- `@Table`, spécifiant le nom de la table associée ;
- `@Id` et `@GeneratedValue`, utilisés pour définir la clé primaire et sa stratégie de génération ;
- `@Column`, permettant de configurer les attributs de la classe ;
- `@ManyToOne` et `@JoinColumn`, utilisées pour représenter les relations entre entités, notamment entre les tables *users* et *roles*.

Cette approche présente l'avantage de centraliser la configuration directement dans le code source, ce qui améliore la lisibilité et réduit le nombre de fichiers nécessaires au fonctionnement de l'application.

En effet, l'utilisation des annotations permet une meilleure cohérence entre la structure du code et la base de données, facilitant ainsi la maintenance et l'évolution du projet. Toutefois, elle introduit une dépendance plus forte entre la logique métier et la configuration de persistance.

Ainsi, bien que moins flexible que la configuration XML dans certains cas avancés, l'approche par annotations constitue aujourd'hui une solution moderne, largement adoptée dans le développement des applications Java.

6. Comparaison entre les configurations XML et Annotations :

Dans le cadre de ce projet, les deux approches de configuration de Hibernate ont été mises en œuvre afin d'en analyser les différences et les avantages respectifs.

La configuration XML se caractérise par une séparation nette entre le code métier et le mapping objet-relationnel. Les informations de persistance sont externalisées dans des

fichiers `.hbm.xml`, ce qui offre une grande flexibilité et permet de modifier le mapping sans toucher au code Java. Cependant, cette approche devient rapidement verbeuse et complexe à maintenir, notamment lorsque le nombre d'entités augmente.

En revanche, la configuration par annotations permet d'intégrer directement le mapping dans les classes Java. Cette approche réduit le nombre de fichiers à gérer et rend le code plus lisible et plus intuitif. Elle facilite également le développement, en permettant au développeur de visualiser immédiatement la correspondance entre les objets et la base de données.

Toutefois, cette intégration du mapping dans le code introduit une dépendance plus forte entre la logique métier et la configuration de persistance, ce qui peut réduire la flexibilité dans certains cas spécifiques.

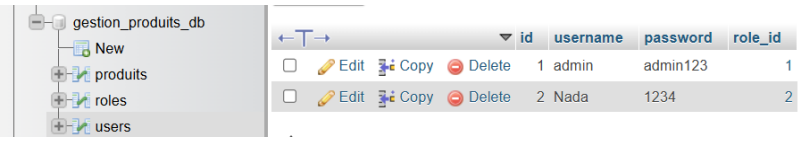
En conclusion, bien que les deux approches soient valides, l'utilisation des annotations apparaît comme la solution la plus adaptée aux pratiques modernes de développement, en raison de sa simplicité, de sa lisibilité et de sa facilité de maintenance.

7. Conception de la base de données :

La base de données utilisée est un système MySQL. Elle est structurée autour de trois tables principales :

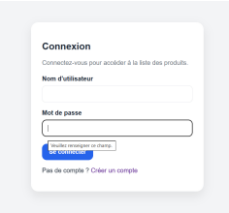
- `users`, contenant les informations des utilisateurs ;
- `roles`, définissant les types d'accès ;
- `produits`, stockant les données relatives aux produits.

Une relation de type **Many-to-One** est établie entre les tables `users` et `roles`, chaque utilisateur étant associé à un rôle unique. Cette modélisation permet de gérer efficacement les droits d'accès et de contrôler les actions autorisées dans l'application.



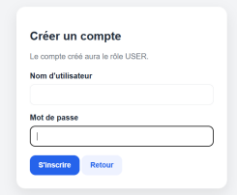
8. Fonctionnalités développées

L'application propose plusieurs fonctionnalités essentielles :



8.1 Authentification :

Un système d'authentification a été mis en place, permettant aux utilisateurs de se connecter à l'aide d'un identifiant et d'un mot de passe. Une session HTTP est créée lors de la connexion afin de maintenir l'état de l'utilisateur.

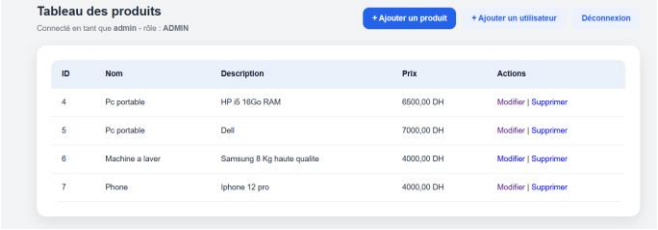


8.2 Gestion des utilisateurs :

Le système permet la création de nouveaux comptes utilisateurs, soit par inscription, soit par l'administrateur. Un mécanisme de vérification garantit l'unicité des noms d'utilisateurs. De plus, chaque utilisateur est associé à un rôle déterminant ses droits d'accès.

8.3 Gestion des produits :

La gestion des produits constitue la fonctionnalité principale de l'application. Les administrateurs disposent de la possibilité d'ajouter, modifier et supprimer des produits, tandis que les utilisateurs standards peuvent uniquement consulter la liste.



9. Sécurisation de l'application :

La sécurité est assurée par l'utilisation de filtres Java EE.

Un filtre d'authentification vérifie si l'utilisateur est connecté avant d'autoriser l'accès aux ressources protégées. Un second filtre restreint certaines opérations aux utilisateurs ayant le rôle administrateur.

Cette approche garantit une séparation claire entre la logique de sécurité et le reste de l'application.

10. Difficultés rencontrées :

Au cours du développement, plusieurs difficultés ont été rencontrées, notamment :

- la configuration correcte des fichiers de mapping XML ;
- la gestion des transactions Hibernate ;

Ces difficultés ont permis de mieux comprendre les mécanismes internes de Hibernate et l'importance d'une bonne structuration du projet.

11. Conclusion :

Ce travail pratique a permis de mettre en œuvre une application web complète en utilisant Java EE et le framework Hibernate, à travers deux approches distinctes de configuration : une première basée sur les fichiers XML, et une seconde reposant sur les annotations Java.

La réalisation de ces deux versions a permis de mieux comprendre les mécanismes de mapping objet-relationnel ainsi que les différences fondamentales entre ces deux approches. La configuration XML, bien qu'offrant une grande flexibilité et une séparation claire entre le code et la persistance, s'est révélée plus complexe et plus difficile à maintenir. En revanche, l'utilisation des annotations a permis de simplifier considérablement le développement, en améliorant la lisibilité du code et en réduisant le nombre de fichiers nécessaires.

Ce projet a également permis de maîtriser l'architecture MVC2, de mettre en place un système d'authentification sécurisé, ainsi que de gérer les droits d'accès selon les rôles des utilisateurs.

Ainsi, cette expérience constitue une base solide pour le développement d'applications web plus avancées, notamment en utilisant des frameworks modernes tels que Spring Boot, qui s'appuient largement sur les annotations.